

In typescript, should I use custom Error types to distinguish between different error cases?

Using custom Error types in TypeScript can be beneficial to distinguish between different error cases, especially in a medium to large-scale application where you might be dealing with a multitude of error scenarios. Here are some points to consider:

Advantages of Using Custom Error Types:

- 1. Clarity and Specificity:** Custom error types make it clear what kind of error occurred. For instance, a `DatabaseError` immediately tells you that something went wrong with the database operations.
- 2. Improved Error Handling:** With custom errors, you can write more specific error handling logic. For example, you might want to handle a `NetworkError` differently from a `ValidationError`.
- 3. Easier Debugging:** Custom error types can carry additional contextual information, making it easier to understand and debug issues. For instance, a `FileNotFoundError` could include the file path that was not found.
- 4. Consistency:** Using a well-defined set of custom error types can make your codebase more uniform and predictable.
- 5. Integration with Error Reporting Tools:** Some error reporting tools can categorize and report errors more effectively if you use distinct error types.

Considerations:

- 1. Complexity:** Introducing custom error types adds a layer of complexity to your code. It's another set of classes/types that developers need to understand and maintain.
- 2. Overhead:** If not used judiciously, you might end up with a proliferation of error types, leading to bloated code and possibly confusing error hierarchies.
- 3. Standardization:** You need to ensure that your custom errors are consistently used and handled across your codebase. Inconsistent usage can lead to confusion and bugs.

Best Practices:

- 1. Extend the Native Error Class:** Ensure your custom errors extend the native `Error` class, so they integrate well with the JavaScript runtime and error-handling mechanisms.
- 2. Include Error Name and Message:** Provide a clear, descriptive message and a unique name for each custom error type.
- 3. Consider Serialization:** If your errors will be sent over the network (e.g., in a backend service), ensure they are serializable and that they maintain useful information after serialization.
- 4. Use a Factory or Constructor for Consistency:** Use a consistent method (like a factory function or a constructor) to create instances of your custom errors. This ensures that they are created with all necessary information.
- 5. Documentation:** Document your custom error types well, explaining when and how they should be used and caught.

In conclusion, using custom error types can be a powerful way to make your error handling more descriptive and context-specific. However, it's important to use them judiciously and maintain consistency and clarity in their implementation.

Give me an example. I have a `fetchData` function. I want to distinguish between a generic error and session expiry

Certainly! Below is an example of how you can define custom error types for a `fetchData` function to distinguish between a generic error and a session expiry scenario in TypeScript.

Step 1: Define Custom Error Classes

First, define custom error classes for the generic error and the session expiry error. Both classes should extend the native `Error` class.

typescript

 Copy code

```
class GenericError extends Error {  
  constructor(message: string) {  
    super(message); // 'message' is a property of Error  
    this.name = 'GenericError';  
  }  
}
```

```

    // Maintains proper stack trace for where our error was thrown (only available in Node.js)
    if (Error.captureStackTrace) {
        Error.captureStackTrace(this, GenericError);
    }
}

class SessionExpiredError extends Error {
    constructor(message: string) {
        super(message);
        this.name = 'SessionExpiredError';
        if (Error.captureStackTrace) {
            Error.captureStackTrace(this, SessionExpiredError);
        }
    }
}

```

Step 2: Implement fetchData Function

Implement the `fetchData` function. Use the custom error classes to throw errors depending on the situation.

typescript

 Copy code

```

async function fetchData(url: string): Promise<any> {
    try {
        const response = await fetch(url);

        // Check for session expiry
        if (response.status === 401) { // 401 is typically used for authentication
            throw new SessionExpiredError('Session has expired. Please log in again');
        }

        // Handle other potential bad statuses
        if (!response.ok) {
            throw new GenericError('An error occurred while fetching data.');
        }

        const data = await response.json();
        return data;
    }
}

```

```
    } catch (error) {  
        // Re-throw the error to be handled by the caller  
        throw error;  
    }  
}
```

Step 3: Handle Errors

When calling `fetchData`, use a try-catch block to handle the specific types of errors.

typescript

 Copy code

```
async function handleDataRequest(url: string) {  
    try {  
        const data = await fetchData(url);  
        console.log('Data fetched successfully:', data);  
    } catch (error) {  
        if (error instanceof SessionExpiredError) {  
            console.error('Session expired:', error.message);  
            // Handle session expiry, for example, redirect to login  
        } else if (error instanceof GenericError) {  
            console.error('A generic error occurred:', error.message);  
            // Handle generic errors  
        } else {  
            console.error('An unexpected error occurred:', error);  
            // Handle unexpected errors  
        }  
    }  
}
```

// Usage example:

```
handleDataRequest('https://api.example.com/data');
```

In this example, the `fetchData` function throws a `SessionExpiredError` if it encounters a session expiry (indicated by a 401 status code). For other types of errors, it throws a `GenericError`. The caller (`handleDataRequest`) then uses a try-catch block to differentiate between these errors and handle them accordingly.

